



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

Lập trình Hướng Đối tượng Object Oriented Programming

\$2 Xây dựng lớp - Đối tượng



Nội dung

- Trừu tượng hóa dữ liệu - lớp đối tượng
- Lớp và thành phần của lớp
- Xây dựng lớp
- Tạo và sử dụng đối tượng
- Thành viên hằng & tĩnh (static)
- Biểu đồ lớp



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

1. Trừu tượng hóa



Trừu tượng hóa

- Abstraction

"a concept or idea not associated with any specific instance"

- Giảm thiểu và tinh lọc các chi tiết nhằm tập trung vào một số khái niệm/vấn đề quan tâm tại một thời điểm.
 - Ví dụ: Các định nghĩa toán học: Ký hiệu x được dùng để thể hiện cho các phép nhân

Trừu tượng hóa

- Trừu tượng hóa điều khiển: Sử dụng các chương trình con (subprogram) và các luồng điều khiển (control flow)
 - Ví dụ: $a := (1 + 2) * 5$
 - Nếu không có trừu tượng hóa điều khiển, LTV phải chỉ ra tất cả các thanh ghi, các bước tính toán mức nhị phân...
- Trừu tượng hóa dữ liệu: Xử lý dữ liệu theo các cách khác nhau
 - Ví dụ: Kiểu dữ liệu
 - Sự tách biệt rõ ràng giữa các thuộc tính trừu tượng của kiểu dữ liệu và các chi tiết thực thi cụ thể của kiểu dữ liệu đó.

Trừu tượng hóa dữ liệu trong LTHĐT

- Đối tượng trong thực tế phức tạp



- Cần đơn giản hóa, bỏ qua những chi tiết không cần thiết
- Chỉ “trích rút” lấy những thông tin liên quan, thông tin quan tâm, quan trọng với bài toán

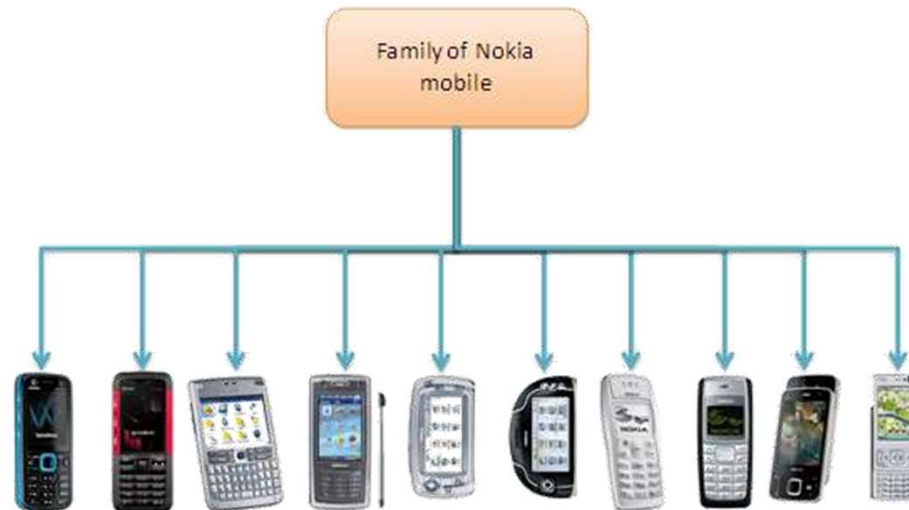
Ví dụ: Điện thoại Nokia



- Những thông tin gì có thể nhận được khi nhìn các "đối tượng" này ?
 - Tất cả là điện thoại Nokia
 - Các điện thoại này có loại nắp trượt, có loại nắp gập, có loại dạng bar
 - Một số điện thoại là dòng doanh nhân, một số dòng âm nhạc, 3G...
 - Bàn phím loại tiêu chuẩn, QWERTY hoặc không có bàn phím
 - Màu sắc, chất liệu, kích cỡ... khác nhau
 - v.v...

Ví dụ: Điện thoại Nokia

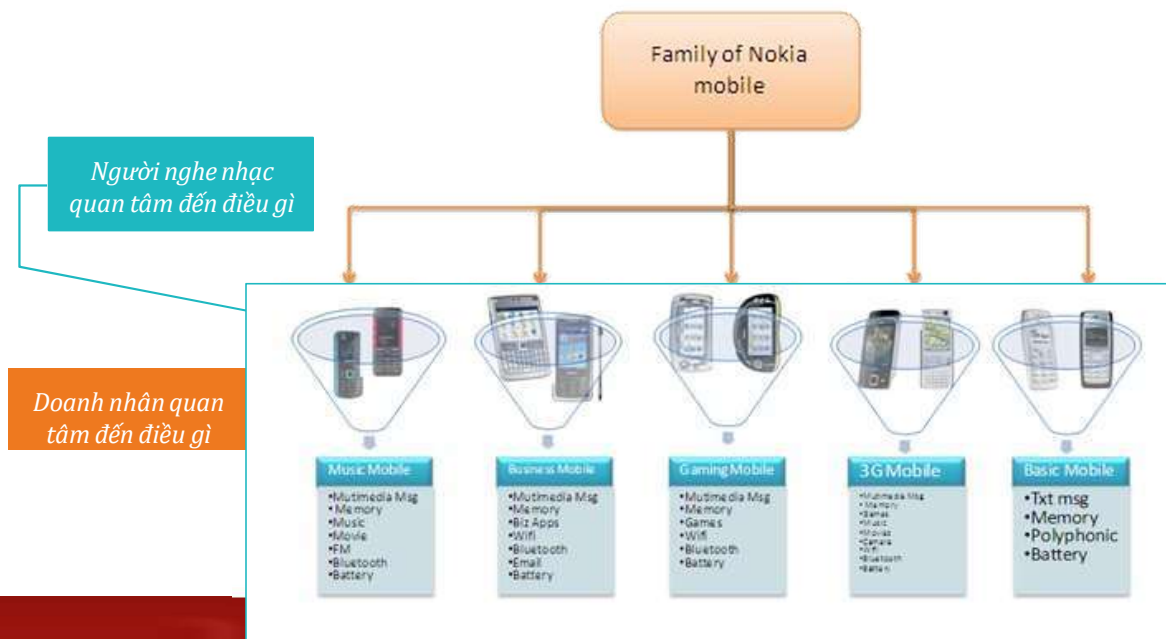
- Tổ chức các đối tượng này theo các đặc tính chung ?



- Coi Tất cả là điện thoại Nokia => Vẫn còn rất khái quát

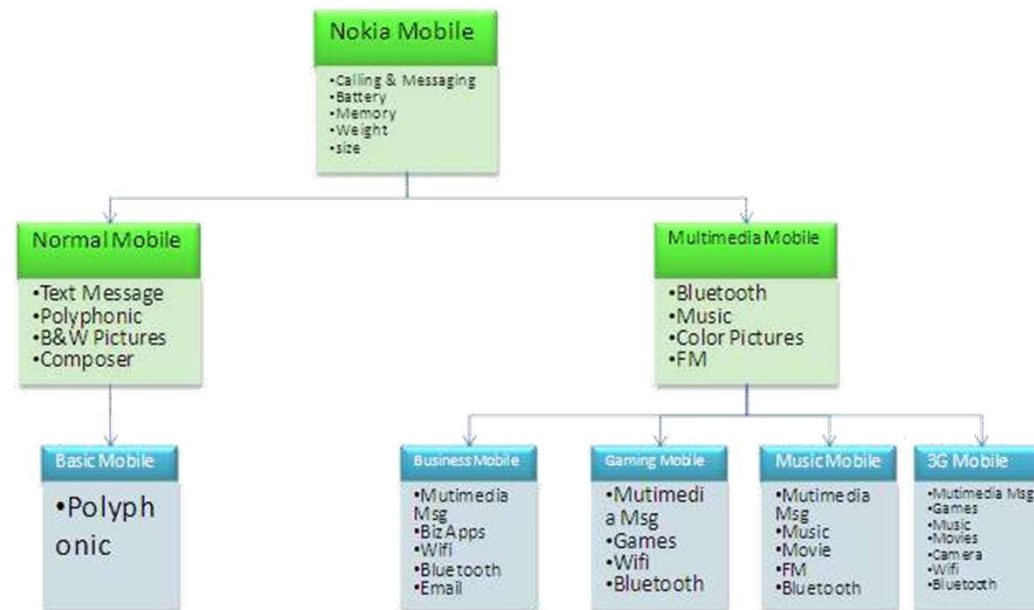
Ví dụ: Điện thoại Nokia

- Chia thành các phân loại chi tiết hơn
 - Ví dụ: Theo chức năng
- Xác định các đặc tính riêng



Ví dụ: Điện thoại Nokia

- Trừu tượng hóa các đối tượng điện thoại Nokia, vd:





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

2. Lớp đối tượng và thành phần của lớp

Lớp đối tượng

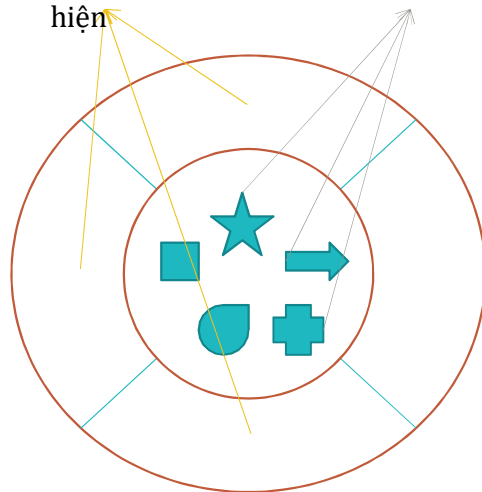
- Lớp (Class) là cách phân loại (classify) các đối tượng dựa trên đặc điểm chung của các đối tượng đó
- Lớp có thể coi là khuôn mẫu/đặc tả để tạo các đối tượng
 - Ví dụ: Người, Sinh Vật, Xe cộ, ...
- Lớp chính là kết quả của quá trình trừu tượng hóa dữ liệu
 - Lớp định nghĩa một kiểu dữ liệu mới, trừu tượng hóa một tập các đối tượng
 - Một đối tượng gọi là một thể hiện của lớp

Các thành phần của lớp

- Lớp đóng gói các phương thức và thuộc tính chung của các đối tượng cùng một loại

Phương thức: các hành vi mà đối tượng có thể thực hiện

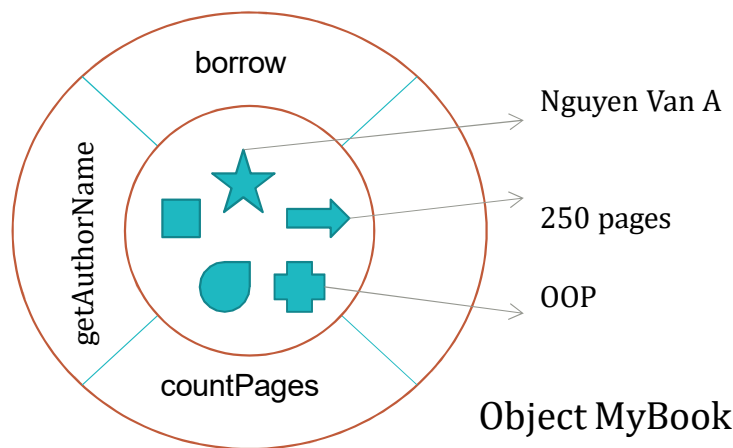
Thuộc tính: các thông tin liên quan đến thuộc tính



Class Book

Thể hiện: Một đối tượng cụ thể

Thuộc tính thể hiện: Những giá trị gán cho các thuộc tính của một đối tượng cụ thể



Object MyBook

Thuộc tính

■ Thuộc tính

- Một thuộc tính của một lớp là một trạng thái chung được đặt tên của tất cả các thể hiện của lớp đó có thể có
- Ví dụ: Lớp Ô tô có các thuộc tính
 - Màu sắc
 - Vận tốc

■ Các thuộc tính của cũng là các giá trị trừu tượng

Mỗi đối tượng có bản sao các thuộc tính của riêng nó

- Ví dụ:
 - Chiếc Ô tô đang đi có thể có màu đen, vận tốc 60 km/h

Phương thức

■ Phương thức

- Xác định các hoạt động/hành vi chung mà tất cả các thể hiện của lớp có thể thực hiện được
- Xác định cách một đối tượng đáp ứng lại một thông điệp nhận được
- Thông thường các phương thức sẽ hoạt động trên các thuộc tính và thường làm thay đổi các trạng thái của lớp
- Bất kỳ phương thức nào cũng phải thuộc về một lớp nào đó

- Ví dụ: Lớp Ô tô có các phương thức
 - Tăng tốc
 - Giảm tốc

Phạm vi (visibility)

- Phạm vi nhìn thấy được: xác định khả năng nhìn thấy được của một thành phần của chương trình với các thành phần khác của chương trình
- Đối với lớp đối tượng
 - Phạm vi nhìn thấy được có thể được áp dụng cho các thành phần của lớp (thuộc tính, Phương thức)
 - Có 2 phạm vi chính:
 - **private**: chỉ truy cập được bên trong lớp đó
 - **public**: có thể truy cập được tại mọi nơi



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

3. Xây dựng lớp



Gói và Không gian tên

- Chương trình lớn => có nhiều lớp => cần tổ chức các lớp đối tượng
- Mục đích:
 - Tổ chức và xác định vị trí lớp, sử dụng phù hợp
 - Tránh xung đột tên
 - Bảo vệ các lớp
- Tùy thuộc nền tảng, ngôn ngữ:
 - Java: gói/package
 - C++, C#: không gian tên/name space

```
package <tên gói>;  
package com.example.mypackage;
```

```
namespace <tên namespace>  
{  
    // Khai báo các lớp ở đây  
}
```

Khai báo lớp

- Là bản mô tả lớp
- Cú pháp: tùy thuộc ngôn ngữ
 - Với Java, C#:

```
<access modifier> class <class name>
{
    //Nội dung class
}
```

Khai báo lớp: có sử dụng chỉ định truy cập

- Chỉ định truy cập: xác định khả năng truy cập từ bên ngoài
- Cơ bản có
 - public: Lớp có thể được truy cập từ bất cứ đâu, kể cả bên ngoài package/namespace chứa lớp đó.
 - private: Lớp chỉ có thể được truy cập trong phạm vi lớp đó

inner class			
	public	private	
Cùng lớp	✓	✓	
Bên ngoài	✓	✓	

Thuộc tính của lớp

- Là các thông tin, trạng thái mà đối tượng của lớp đó có thể mang
- Các thuộc tính phải được khai báo bên trong lớp
- (thông thường) Mỗi đối tượng có bản sao các thuộc tính của riêng nó
=> Giá trị của cùng một thuộc tính của các đối tượng khác nhau là khác nhau.
- Bản chất của các thuộc tính là các thành phần dữ liệu của đối tượng
 - Khai báo: tương tự như biến

Thuộc tính

- Cú pháp khai báo thuộc tính (như java, C#)

`<access modifier> <kiểu> <tên thuộc tính> ;`

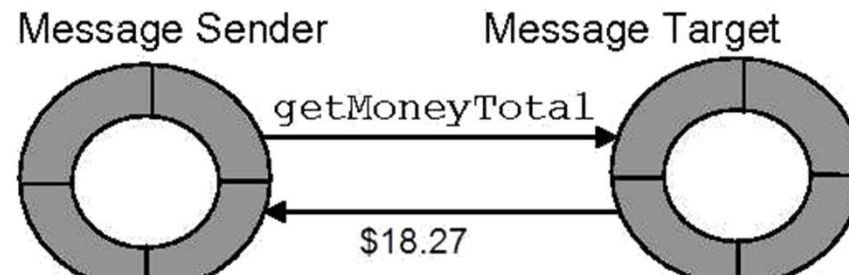
- Thuộc tính có thể được khởi tạo khi khai báo
 - Các giá trị mặc định sẽ được sử dụng nếu thuộc tính không được khởi tạo

```
public string StudentCode;
```

```
public string StudentCode { get; set; }
```

Phương thức

- Phương thức: Xác định cách một đối tượng đáp ứng lại thông điệp
- Khai báo Phương thức: tương tự khai báo hàm
- Phương thức xác định các hoạt động của lớp
- Phương thức phải thuộc về một lớp nào đó



Phương thức / method

- Cú pháp (java, C#)

```
<access modifier> <Tên Phương thức> (<các tham số>
{
    // Nội dung cài đặt
}
}
```

Chỉ định truy cập: cho thành viên lớp (property, method)

- Chỉ định truy cập cho thành viên của lớp: tương tự với lớp
 - public: Thuộc tính hoặc phương thức có thể được truy cập từ bất cứ đâu, kể cả từ bên ngoài
 - private: Thuộc tính hoặc phương thức chỉ có thể được truy cập trong phạm vi lớp đó

Ví dụ lớp

```
public class BankAccount
{
    public string owner;
    public double balance;
    public bool debit(double amount) {
        if (amount > balance)
            return false;
        else
        {
            balance -= amount;
            return true;
        }
    }
    public void credit(double amount) => balance += amount; //action body style
}
public void credit(double amount) {
    balance += amount;
}
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

4. Tạo và sử dụng đối tượng

Vòng đời:

khai báo, khởi tạo

sử dụng

giải phóng

Khai báo và khởi tạo dữ liệu

- Trong nhiều ngôn ngữ HDT (C++, Java, C#): mọi dữ liệu cần phải được khai báo và khởi tạo trước khi sử dụng
- Ví dụ
 - Khai báo
`int i;`
 - Khởi tạo
`i = 3;`
 - Hoặc có thể kết hợp khai báo và khởi tạo
`int i = 3;`

Khai báo và khởi tạo dữ liệu



■ Tự động khởi tạo

- Trong một số ngôn ngữ HDT như C++, C#: dữ liệu có thể tự động được khởi tạo khi khai báo

Ví dụ: `int i;`

Dữ liệu sẽ được khởi tạo với giá trị mặc định của kiểu dữ liệu tương ứng

- Trong một số ngôn ngữ như Java: dữ liệu KHÔNG được tự động khởi tạo khi khai báo
- Ví dụ:

`int i;`

`System.out.println(i); //LỖI`

Khai báo

- Để khai báo dữ liệu, ta cần phải xác định kiểu của dữ liệu đó
 - Cú pháp thường là: `<kiểu> tên biến;`
 - Ví dụ: `int i; //Biến i kiểu int`
- Trong OOP, lớp có thể coi là kiểu dữ liệu trừu tượng do người dùng định nghĩa và đối tượng chính là biến của kiểu dữ liệu đó
 - => Khai báo: tương tự khai báo biến
 - Cú pháp thường là : `<Tên Lớp> tên đối tượng;`
 - Ví dụ:
`BankAccount acc; //Đối tượng acc là đối tượng kiểu BankAccount`

Khởi tạo

- Đối với kiểu dữ liệu nguyên thủy: Dùng toán tử =
 - Ví dụ: `i = 3;`
- Đối với kiểu dữ liệu tham chiếu (1): Khởi tạo bằng toán tử **new**
 - Ví dụ: `acc1 = new BankAccount();`
- Nếu không được khởi tạo: đối tượng mang giá trị **null**

(1) Có thể xem tương tự con trỏ trong C

Khởi tạo

- Khi đối tượng được khởi tạo, các thành phần dữ liệu (thuộc tính) của đối tượng được khởi tạo với giá trị mặc định của kiểu dữ liệu tương ứng, vd:
 - number data type => gán 0;
 - reference type => gán null
 - boolean => gán false

Kết hợp khai báo và khởi tạo dữ liệu

- Có thể kết hợp khai báo và khởi tạo dữ liệu

- Cú pháp:

<Tên Lớp> tên đối tượng = new <Tên Lớp>();

Ví dụ:

BankAccount acc = new BankAccount();

Truy cập đến phương thức và thuộc tính của đối tượng

- Sử dụng toán tử `< . >`

```
public class Student
{
    public string StudentCode { get; set; }
    public string FullName { get; set; }
    public DateTime Birthday { get; set; }

    public Student(string studentCode, string fullName, DateTime
birthday)
    {
        StudentCode = studentCode;
        FullName = fullName;
        Birthday = birthday;
    }
}
```

```
Student sv1 = new Student("20241234", "Nguyễn Văn A", new DateTime(2006, 1, 1));
Console.WriteLine($"Sinh viên {sv1.FullName} có mã {sv1.StudentCode} SN {sv1.Birthday}");
```

Tự tham chiếu

- Từ khóa **this**
- Tham chiếu đến thể hiện (instance) hiện tại của lớp
- Tránh nhập nhằng giữa biến cục bộ, tham số, thành phần dữ liệu

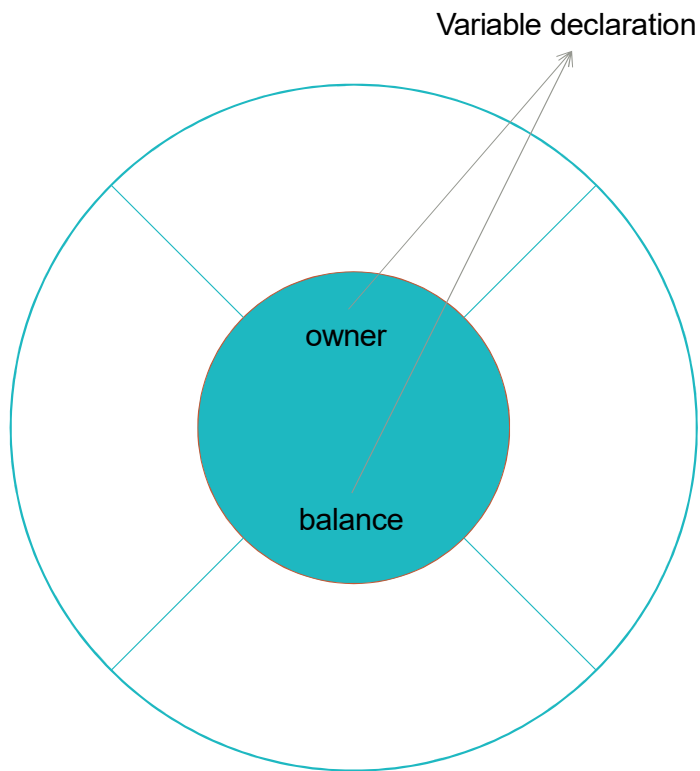
```
public class Account
{
    private int accID;
    private string accName;
    private double accBalance;

    // Constructor
    public Account(int accID, string accName,
double accBalance)
    {
        this.accID = accID;
        this.accName = accName;
        this.accBalance = accBalance;
    }
}
```

Ví dụ: từ khóa **this** (java)

```
public class Account {  
    // instance variable  
    String owner; // Account name  
    long balance; // Balance  
    //...  
    // value setting method  
    void setAccountInfo(String owner, long balance) {  
        this.owner = owner;  
        this.balance = balance;  
    }  
    //...  
}
```

Ví dụ: Khởi tạo đối tượng



Class Account

Account object of Mrs. Giang



Account object of Mr. Tuan



Ví dụ: Khởi tạo đối tượng

```
public class Account
{
    public int accID;
    public string accName;
    public double accBalance;
    public Account(int accID, string accName, double
accBalance)
    {
        this.accID = accID;
        this.accName = accName;
        this.accBalance = accBalance;
    }
    // Method to display the account information
    public void Display()
    {
        Console.WriteLine($"Account ID: {accID}, Balance:
{accBalance}, Name: {accName}");
    }
    // Method to increase the balance
    public void Deposit(double amount)
    {
        if (amount > 0)
        {
            accBalance += amount;
        }
    }
}
```

```
Account acc1 = new Account();
Account acc2 = new Account();
```

Ví dụ: Truy cập đến các thuộc tính

```
//caller class: class use Account obj  
Account acc1 = new Account();  
acc1.accID = 101010192;  
acc1.accName = "Nguyen Huong Giang";  
acc1.accBalance = 2000000;
```

```
Account acc2 = new Account();  
acc2.accID = 101015214;  
acc2.accName = "Nguyen Manh Tuan";  
acc2.accBalance = 1000000;
```

Account object of Ms. Giang



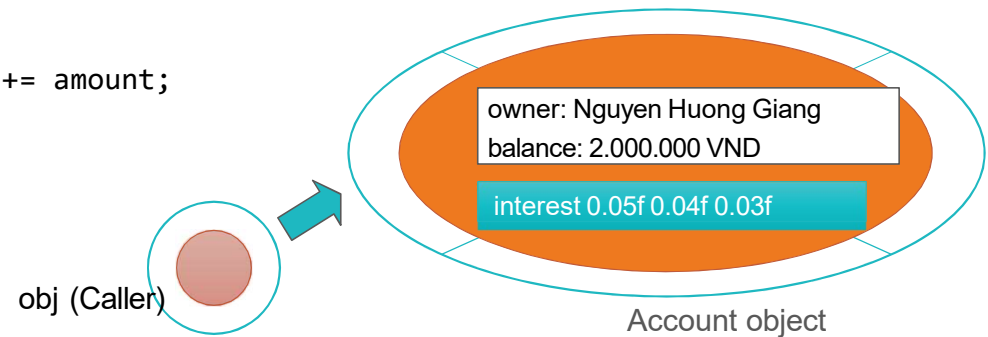
Account object of Mr. Tuan



Ví dụ: Truy cập đến các phương thức

```
public class Account
{
    public int accID;
    public string accName;
    public double accBalance;
    // Method to display the account information
    public void Display()
    {
        Console.WriteLine($"ID: {accID}, Balance: {accBalance}, Name: {accName}");
    }
    // Method to increase the balance
    public void Deposit(double amount)
    {
        if (amount > 0)
        {
            accBalance += amount;
        }
    }
}
```

```
//caller class: class use Account obj
acc1.Display();
acc1.Deposit(1000000);
acc1.Display();
```



Tham chiếu đến lớp khác namespace (.net)/gói (java)

- Nếu lớp cùng không gian tên (namespace hay package) => tham chiếu chỉ cần tên lớp

```
Account acc1;
```

- Nếu khác không gian tên =>

- Cần tên đầy đủ (qualified name) - namespace + class name

```
Banking.Account acc1 = new Account();  
System.Console.WriteLine($"Số dư tài khoản sau khi nạp tiền: {acc1.accBalance}");
```

- Hoặc khai báo (using hoặc import)

```
using System;  
using CS01HelloWorld.Banking;
```

Đối tượng trong C++ tham khảo



- Trong C++, có 2 cách sử dụng đối tượng
 - Khởi tạo đối tượng truyền thống

```
BankAccount acc;  acc.deposit(1000000);
```

- Con trỏ đối tượng ← tương tự đối tượng trong Java

```
BankAccount *acc = new BankAccount(); acc->deposit(1000000);
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

5 Thành viên tĩnh & thành viên hằng

static

final (java)

Thành viên tĩnh (static member)

- (nhắc lại) Trong lập trình cấu trúc: biến cục bộ (local) trong thủ tục
 - Không khai báo static
 - Khai báo static: khởi tạo 1 lần

Biến địa phương không static

```
void f()  
{ int x=0;  
    x++;  
}
```

Lần gọi 1: f() x= 0

Lần gọi 2: f() x= 0

Biến địa phương khai báo static

```
void f()  
{ static int x=0;  
    x++;  
}
```

Lần gọi 1: f() x= 0

Lần gọi 2: f() x= 1

Thành viên tĩnh (static member) trong HĐT

- Trong lập trình hướng đối tượng
 - Các thành viên bình thường là thành viên thuộc về đối tượng cụ thể (mỗi đối tượng chứa bản sao thành viên)

```
public class Student
{
    //
    public string StudentCode { get; set; }
    public string FullName { get; set; }
    public DateTime Birthday { get; set; }

    public Student(string studentCode, string fullName, DateTime
birthday)
    {
        StudentCode = studentCode;
        FullName = fullName;
        Birthday = birthday;
    }
}
```

```
Student sv1 = new Student("PH001", "Nguyễn Văn A", new DateTime(2002, 1, 1));
Student sv2 = new Student("PH002", "Trần Văn B", new DateTime(2002, 10, 10));
Console.WriteLine($"Student sv1: {sv1.StudentCode} {sv1.FullName}");
Console.WriteLine($"Student sv2: {sv2.StudentCode} {sv2.FullName}");
```

Thành viên tĩnh (static member) trong HĐT

- Trong lập trình hướng đối tượng
 - Các thành viên bình thường là thành viên thuộc về đối tượng cụ thể
 - Các thành viên tĩnh (static) là các thành viên thuộc về lớp

```
Employee.NextId = 2024000;  
Employee emp1 = new Employee("Nguyen", "Van A");  
Console.WriteLine($"Employee: {emp1.FirstName} {emp1.LastName} {emp1.Id}");
```

```
class Employee  
{  
    public static int NextId;  
    public int Id { get; set; }  
    public string FirstName { get; set; }  
    public string LastName { get; set; }  
  
    public Employee(string firstName, string lastName)  
    {  
        FirstName = firstName;  
        LastName = lastName;  
        Id = NextId;  
        NextId++;  
    }  
}
```

- Cú pháp khai báo thành viên static:

```
<Access modifier> static <type> <member>;
```

Thành viên tĩnh

Thuộc tính static

- Là thuộc tính mang thông tin chung của một lớp
- Thay đổi giá trị của một thành viên static trong một đối tượng của lớp sẽ thay đổi giá trị của thành viên này của tất cả các đối tượng khác của lớp đó.

Các loại thành viên tĩnh (C#)

- **Biến tĩnh (static variable)**
- Phương thức tĩnh (static method)
- Phương thức khởi tạo tĩnh (static constructor)
- Lớp tĩnh (static class)

```
public class Student
{
    //
    public string StudentCode { get; set; }
    public string FullName { get; set; }
    public DateTime Birthday { get; set; }
    static public string SchoolName = "HUST";

    public Student(string studentCode, string
fullName, DateTime birthday)
    {
        StudentCode = studentCode;
        FullName = fullName;
        Birthday = birthday;
    }
}
```

Ví dụ (Java => có khác biệt C#

```
class TestStatic{
    public static int iStatic; public int iNonStatic;
}

public class TestS {
    public static void main(String[] args) {
        TestStatic obj1 = new TestStatic();
        obj1.iStatic = 10; obj1.iNonStatic = 11;
        System.out.println(obj1.iStatic + "," + obj1.iNonStatic);
        TestStatic obj2 = new TestStatic();
        System.out.println(obj2.iStatic + "," + obj2.iNonStatic);
        obj2.iStatic = 12;
        System.out.println(obj1.iStatic + "," + obj1.iNonStatic);
    }
}
```

```
10,11
10,0
12,11
```

Thành viên tĩnh

Phương thức static

Các loại thành viên tĩnh (C#)

- Biến tĩnh (static variable)
- **Phương thức tĩnh (static method)**
- Phương thức khởi tạo tĩnh (static constructor)
- Lớp tĩnh (static class)

- Các phương thức không tương tác với các "thể hiện" của lớp
- Các phương thức "tiện ích", không cần thiết phải khởi tạo đối tượng để sử dụng
- Các phương thức static chỉ có thể truy cập vào các thuộc tính static và gọi các phương thức static trong cùng lớp

```
class ConsoleHelper  
{
```

```
    public static void WriteLine(object message,  
        ConsoleColor fgColor = ConsoleColor.White,  
        ConsoleColor bgColor = ConsoleColor.Black)  
    {  
        Console.ForegroundColor = fgColor;  
        Console.BackgroundColor = bgColor;  
        Console.WriteLine(message);  
    }  
}
```

```
static void Main(string[] args) ...  
    // gọi phương thức tĩnh  
    ConsoleHelper.WriteLine("Hello world!", ConsoleColor.Magenta,  
        ConsoleColor.White);
```

Ví dụ (Java) - tham khảo

```
class MyUtils {  
    public static double mean(int[] p) {  
        int sum = 0;  
        for (int i=0; i<p.length; i++) {  
            sum += p[i];  
        }  
        return ((double)sum) / p.length;  
    }  
}  
  
// Gọi phương thức tính bên trong lớp  
double avgAtt = mean(attendance);  
// Gọi phương thức tính bên ngoài lớp  
double avgAtt = MyUtils.mean(attendance);
```

Phương thức mean có thể không phải khai báo static tuy nhiên khi đó muốn gọi nó phải thông qua một đối tượng

Thành viên tĩnh

Phương thức khởi tạo static

Các loại thành viên tĩnh (C#)

- Biến tĩnh (static variable)
- **Phương thức tĩnh (static method)**
- Phương thức khởi tạo tĩnh (static constructor)
- Lớp tĩnh (static class)

- Phương thức được thực thi trước khi bất kỳ một object nào đc tạo
- Không có điều khiển truy cập
- Không có tham số
- Thường dùng để thiết lập giá trị các thuộc tính (static) cần tính toán phức tạp

```
class Employee
{
    public static int NextId; //đúng ra là readonly
    static Employee()
    {
        //get next ID based on current ID from
        database
        Random randomGenerator = new Random();
        NextId = randomGenerator.Next(101, 999);
    }
    // ...
}
```

//caller

```
Employee emp1 = new Employee("Nguyen", "Van A");
Console.WriteLine($"Employee: {emp1.FirstName} {emp1.LastName} {emp1.Id}");
```

Thành viên tĩnh

Lớp static

- Lớp chỉ có các thành phần static, không có thể hiện => không có khai báo biến, không có khởi tạo
- Thường là đóng gói các tiện ích (helper), hằng số

```
int[] numbers = new[] { 1, 2, 3, 4, 5, 6 };
int max = MathLib.Max(numbers);
//System.Console is a also static class
Console.WriteLine($"Max value: {max}");
```

Các loại thành viên tĩnh (C#)

- Biến tĩnh (static variable)
- **Phương thức tĩnh (static method)**
- Phương thức khởi tạo tĩnh (static constructor)
- Lớp tĩnh (static class)

```
public static class MathLib
{
    public static int Max(params int[] numbers)
    {
        int max = numbers[0];
        foreach (var number in numbers){
            if (number > max)
                max = number;
        }
        return max;
    }
    public static int Min(params int[] numbers)
    {
        //...
    }
}
```

So sánh Thành viên lớp và thành viên đối tượng

- Thành viên đối tượng
 - Thuộc tính/phương
 - thức chỉ được truy cập
 - thông qua đối tượng
 - Mỗi đối tượng có 1 bản sao riêng của 1 thuộc tính đối tượng
 - Giá trị của 1 thuộc tính đối tượng của các đối tượng khác nhau là khác nhau.
- Thành viên lớp (static)
 - Thuộc tính/phương
 - thức có thể được truy
 - cập thông qua lớp
 - Các đối tượng có chung 1 bản sao của 1 thuộc tính lớp
 - Giá trị của 1 thuộc tính lớp của các đối tượng khác nhau là giống nhau.



Ví dụ (java)

- Lớp JOptionPane trong javax.swing
 - *Thuộc tính*
 - *Phương thức*

Ví dụ (java)

```
JOptionPane.showMessageDialog(null,  
    "Ban da thao tac loi", "Thong bao loi",  
    JOptionPane.ERROR_MESSAGE);
```



```
JOptionPane.showConfirmDialog(null,  
    "Ban co chac chan muon thoat?",  
    "Hay lua chon", JOptionPane.YES_NO_OPTION);
```



Ví dụ (java)

```
Object[] options = { "OK", "CANCEL"};  
JOptionPane.showOptionDialog(null,  
    "Nhan OK de tiep tuc", "Canh bao",  
    JOptionPane.DEFAULT_OPTION,  
    JOptionPane.WARNING_MESSAGE,  
    null, options, options[0]);
```



Thành viên hằng => java hỗ trợ, C# không cài đặt

- Một thuộc tính/phương thức không thể thay đổi giá trị/nội dung trong quá trình sử dụng.
- Cú pháp khai báo: Sử dụng từ khóa **final**
<chỉ định truy cập> final <kiểu> tên hằng = giá trị;
- Ví dụ:

```
final double PI = 3.141592653589793;
public final int VAL_THREE = 39;
private final int[] A = { 1, 2, 3, 4, 5, 6 };
```

Thành viên hằng & tĩnh => java hỗ trợ

Thông thường các hằng số liên quan đến lớp được khai báo là **static final**

- Tương đương với **const** trong C++

```
public class MyDate {  
    public static final long SECONDS_PER_YEAR =  
        31536000;  
    ...  
}  
...  
long years = MyDate.getMillisSinceEpoch() /  
    (1000*MyDate.SECONDS_PER_YEAR);
```

Extension method

- Cho phép "chèn" phương thức của lớp này vào làm phương thức thành viên của lớp khác
- Tham số có từ khóa this theo sau là kiểu dữ liệu
- Có thể gọi qua kiểu dữ liệu => ẩn
- Có thể gọi như phương thức static thông thường

```
static class ExtensionMethods
{
    public static void ToConsole(this string message)
    {
        Console.WriteLine(message);
    }
    public static int ToInt(this string number)
    {
        return int.Parse(number);
    }
}
```

```
"Hello world!".ToConsole();
int i = "2000".ToInt();
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

6. Biểu đồ lớp



Biểu đồ lớp

- Lớp (class) được biểu diễn bằng 1 hình chữ nhật với 3 vùng
 - Tên lớp
 - Cấu trúc (thuộc tính)
 - Hành vi (phương thức)

Professor
- name - employeeID : UniqueId - hireDate - status - discipline - maxLoad
+ submitFinalGrade() + acceptCourseOffering() + setMaxLoad() + takeSabbatical() + teachClass()

Chỉ định truy cập

- Trong biểu đồ lớp
- Dấu - được sử dụng để thể hiện chỉ định truy cập private
- Dấu + được sử dụng để thể hiện chỉ định truy cập public

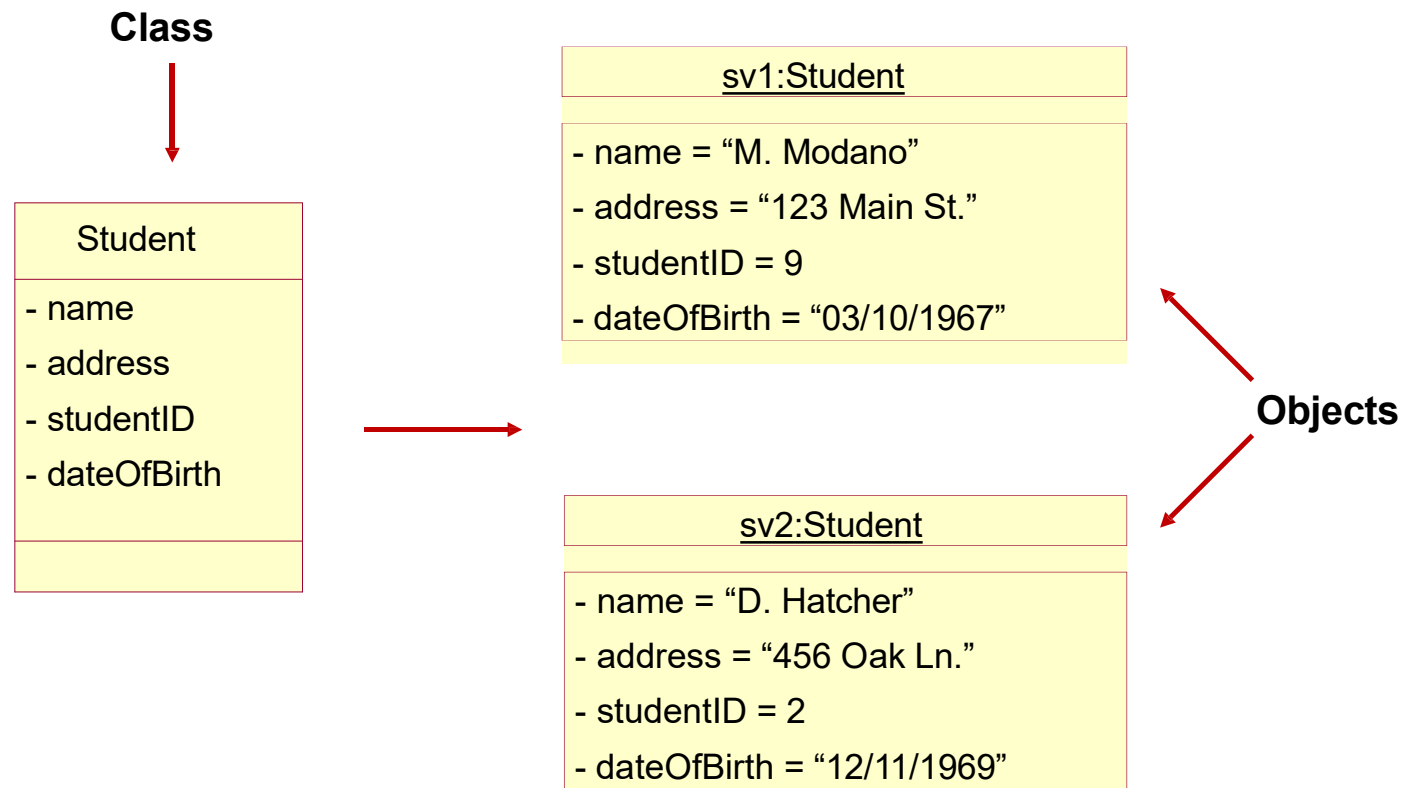
Professor
- name - employeeID : UniqueId - hireDate - status - discipline - maxLoad
+ submitFinalGrade() + acceptCourseOffering() + setMaxLoad() + takeSabbatical() + teachClass()

Giá trị tĩnh

- Giá trị tĩnh: Thể hiện bằng dấu gạch chân

Student
- name - address - studentID - <u>nextAvailID</u> : int
+ addSchedule ([in] theSchedule : Schedule, [in] forSemester : Semester) + getSchedule ([in] forSemester : Semester) : Schedule + hasPrerequisites ([in] forCourseOffering : CourseOffering) : boolean # passed ([in] theCourseOffering : CourseOffering) : boolean + <u>getNextAvailID</u> () : int

Lớp và đối tượng trong UML



Ví dụ

- Lớp BankAccount

BankAccount
- owner: String - balance: double
+ debit(double): boolean + credit(double)

- Lớp Student

Student
- name - address - studentID - dateOfBirth